

HANDS_ON_Lab2_WaitNotify_nur_Files

Lab F4: Wait / Notify ohne Datenbank (file-only)

Apache NiFi 2.8.0, Single-User-Login (`admin`), UI unter <https://localhost:8443/nifi>
File-only-Gegenstück zu “Lab 2: Wait/Notify (DB)” aus `HANDS_ON_Abhaengigkeiten.md`.
Kein DBCPConnectionPool, kein SQLite-Treiber nötig. Fertiges Referenz-Lab auf der Canvas: “**Lab F4 - Wait-Notify (file)**” (Werte hier aus der laufenden Mac-Instanz ausgelesen.)

Lernziel (identisch zu Lab 2): Zwei unabhängige Flows so koppeln, dass Job B erst nach Abschluss von Job A startet, ohne dass B von A direkt FlowFiles bekommt. Das NiFi-Pendant zu “B depends on A”.

Unterschied zur DB-Variante: Statt “CSV in DB laden” archiviert Job A die Datei (PutFile); statt “Zeilen per SQL zählen” schreibt Job B nach Freigabe eine Marker-Datei. Der **Wait/Notify-Teil ist identisch** : darum geht es in diesem Lab.

Pfade (echte Werte des Canvas-Labs)

```
Input A:  nifi-2.8.0/data/input/labF4          <- hier Datei einwerfen
Archiv A:  nifi-2.8.0/data/archive              <- Job A legt verarbeitete Datei ab
Output B:  nifi-2.8.0/data/output/freigegeben  <- Job B Marker nach Freigabe
```

Windows-VM (eigene NiFi): analog mit `C:/nifi/data/...` und Forward-Slashes in PutFile.

Controller Services (einmalig, beide aktivieren)

Das ist ein **Signal-Cache**, keine Datenbank. Im Add-Dialog nach `MapCache` filtern.

1. `MapCacheServer` (Defaults, Port 4557) → aktivieren.
2. `MapCacheClientService`: Server Hostname = `localhost`, Server Port = `4557` → aktivieren.

Reihenfolge: erst Server, dann Client. (Im Canvas-Lab teilen Wait und Notify denselben Client.)

Flow (wie auf der Canvas)

```
Job A:  GetFile —success—> PutFile archive —success—> Notify   (Signal: dateiX-
fertig)
Job B:  GenerateFlowFile —success—> Wait —success—> PutFile freigegeben
      └─ wait (Self-Loop zurück auf Wait)
```

Job A: “verarbeiten und melden”

A: GetFile - Input Directory = `.../data/input/labF4` - Keep Source File = `false` - Polling Interval = `2 sec` empfohlen (Default 30s → sonst lange Wartezeit)

A: PutFile archive (ersetzt PutDatabaseRecord) - Directory = `.../data/archive` - Create Missing Directories = `true`, Conflict Resolution = `replace` - `failure` terminieren, `success` → Notify.

A: Notify - Release Signal Identifier = `dateiX-fertig` - Distributed Cache Service = der MapCacheClientService - `success` und `failure` terminieren.

| Job A archiviert die Datei und setzt **danach** das Signal `dateiX-fertig`.

Job B: “warten, dann Marker”

B: GenerateFlowFile (Trigger) - Scheduling → Run Schedule = `10 sec` (regelmäßiger “Versuch”, leere FlowFile genügt)

B: Wait - Release Signal Identifier = `dateiX-fertig` - Distributed Cache Service = derselbe MapCacheClientService - Target Signal Count = `1` - Relationships: - `success` → PutFile freigegeben - `wait` → **Self-Loop zurück auf Wait** (Pflicht!) - `expired` → terminieren (oder PutFile “timeout”) - `failure` → terminieren

B: PutFile freigegeben (ersetzt ExecuteSQL + PutFile) - Directory = `.../data/output/freigegeben` - Create Missing Directories = `true`, Conflict Resolution = `replace` - `success` / `failure` terminieren.

Verifikation / Aha-Effekt

1. Starte **nur Job B**. Die FlowFiles stauen sich auf der `wait`-Schleife, im Output passiert nichts:

```
ls -l data/output/freigegeben/      # leer -> B wartet auf A
```

2. Starte **Job A** und wirf eine Datei ein:

```
cp data/samples/orders.csv data/input/labF4/
```

Sobald Notify `dateiX-fertig` setzt, lässt Wait durch:

```
ls -l data/archive/                # Job A: archivierte Datei
ls -l data/output/freigegeben/     # Job B: Marker nach Freigabe
```

Praxis-Hinweise (gelten wie bei Lab 2)

1. **wait-Self-Loop ist Pflicht.** Die `wait`-Relationship MUSS auf Wait selbst zurückgeschleift werden, sonst verschwinden wartende FlowFiles. NUR `expired/`
`failure` terminieren, **nicht** `wait`.
 2. **~30s Geduld:** FlowFiles auf der `wait`-Schleife werden penalized (Default 30s); Freigabe nach dem Signal kann bis zu 30s dauern.
 3. **Signal-Reset zwischen Läufen:** Signal bleibt im MapCache. Sauberer Neustart: `MapCacheServer` kurz deaktivieren/aktivieren.
-

Optional: echten Zeilen-Report wie in der DB-Variante, aber ohne DB

Soll Job B die **Zeilenzahl** der Datei schreiben (wie ExecuteSQL `COUNT(*)`), record-basiert ohne DB:

```
... Wait —success—> FetchFile —> QueryRecord —> PutFile (Report mit Anzahl)
```

- **FetchFile:** File to Fetch = konkreter Pfad der archivierten Datei.
- **QueryRecord** (CSVReader + CSVRecordSetWriter): dynamische Property `anzahl` = `SELECT COUNT(*) AS anzahl FROM FLOWFILE`. Exaktes SQL-Äquivalent, nur auf der Datei.
- **PutFile:** Ergebnis nach `data/output/freigegeben`.

Für die reine Wait/Notify-Demo reicht der Marker-PutFile.

Reset

```
rm -f data/input/labF4/* data/archive/* data/output/freigegeben/*  
# Signal zurücksetzen: MapCacheServer kurz deaktivieren/aktivieren
```

Häufige Stolpersteine

- **Wait lässt nie durch:** Notify und Wait nutzen unterschiedlichen Signal-Namen oder Cache-Client. Beide müssen exakt `dateiX-fertig` + derselbe Client sein.
- `wait` **nicht im Self-Loop:** wartende FlowFiles verschwinden → kein Marker.
- **Nichts im Output trotz Job A:** bis zu 30s Penalty abwarten; Signal evtl. schon verbraucht (Reset).
- **PutFile schreibt nicht (Windows):** Backslashes im Pfad oder Ordner fehlt; Forward-Slashes + `Create Missing Directories = true`.